

Introduction to Data Science — Final Exam Answer Key

SECTION A — Short Answer (5 Marks Each, Do Any 4)

Q1. Define Nominal, Ordinal, and Binary attributes. How do they differ from Numeric attributes?

Attribute Type	Definition	Example
Nominal	Categories with no inherent order . Values are just labels.	Colour (Red, Blue, Green), Gender (Male, Female)
Ordinal	Categories with a meaningful order/ranking , but differences between ranks are not measurable.	Education level (High School < Bachelor's < Master's < PhD), Rating (Poor, Average, Good, Excellent)
Binary	A special case of nominal with only two possible values (0/1 or True/False).	Smoker (Yes/No), Test Result (Pass/Fail)
Numeric	Quantitative values on which arithmetic operations are meaningful. Can be Discrete (countable: no. of students) or Continuous (measurable: temperature, weight).	Age = 25, Salary = ₹50,000, Height = 5.8 ft

Key Difference: Nominal, Ordinal, and Binary are **qualitative** (categorical) — you cannot perform mathematical operations on them. Numeric attributes are **quantitative** — you can compute mean, sum, standard deviation, etc.

Q2. What is Data Normalization? Explain Min-Max normalization with example.

Data Normalization is the process of scaling numeric data to a standard range (typically 0 to 1) so that no single feature dominates others due to its scale. It is essential before applying distance-based algorithms like k-NN or K-Means.

Min-Max Normalization Formula:

X_normalized = (X - X_min) / (X_max - X_min)

Numerical Example:

Given dataset: {10, 20, 30, 40, 50}

- X_min = 10, X_max = 50

Original (X)	Normalized
10	(10 - 10) / (50 - 10) = 0.00
20	(20 - 10) / (50 - 10) = 0.25
30	(30 - 10) / (50 - 10) = 0.50
40	(40 - 10) / (50 - 10) = 0.75
50	(50 - 10) / (50 - 10) = 1.00

All values are now scaled to the range [0, 1].

Q3. Differentiate between Supervised and Unsupervised Learning.

Aspect	Supervised Learning	Unsupervised Learning
Definition	Model learns from labelled data (input-output pairs).	Model learns from unlabelled data (no target variable).
Goal	Predict the output for new inputs.	Discover hidden patterns or groupings in data.

Training Data	Requires labelled dataset.	No labels needed.
Feedback	Direct feedback through known correct answers.	No feedback; model finds structure on its own.
Types	Classification, Regression	Clustering, Association, Dimensionality Reduction

Two Supervised Learning Algorithms:

- 1. **k-Nearest Neighbours (k-NN)** — Classifies based on majority vote of nearest neighbours.
- 2. **Linear Regression** — Predicts continuous output by fitting a line to data.

Two Unsupervised Learning Algorithms:

- 1. **K-Means Clustering** — Partitions data into K clusters based on distance.
- 2. **Apriori Algorithm** — Finds frequent itemsets and association rules in transactional data.

Q4. Techniques used in Data Cleaning. Handling missing values and noisy data.

Data Cleaning is the process of detecting and correcting (or removing) corrupt, inaccurate, or irrelevant records from a dataset.

Key Techniques:

- 1. **Handling Missing Values:**
 - **Deletion:** Remove rows/columns with missing values (suitable when missing % is small).
 - **Imputation:** Fill with mean (numeric), median (skewed), or mode (categorical).
 - **Forward/Backward Fill:** Use the previous or next valid value (useful in time-series).
 - **Predictive Imputation:** Use ML models to predict missing values.
- 2. **Handling Noisy Data:**
 - **Binning:** Sort values and smooth by replacing with bin mean/median/boundary values.
 - **Regression:** Fit data to a regression function to smooth noise.
 - **Clustering:** Group data and identify outliers that don't fit any cluster.
 - **Manual Inspection:** Domain experts review and correct values.
- 3. **Other Tasks:** Removing duplicates, resolving inconsistencies (e.g., "NY" vs "New York"), standardizing formats (date formats, units).

Why it matters: Clean data ensures better model accuracy, avoids biased results, and produces reliable statistical inferences.

Q5. What is a CSV file? Python code to read, display shape, and list columns.

CSV (Comma-Separated Values) is a plain text file format where each row represents a data record, and fields within each record are separated by commas. It is widely used for data exchange because of its simplicity and compatibility with most tools (Excel, Python, R, databases).

```
import pandas as pd

# Read the CSV file
df = pd.read_csv('students.csv')

# Display the shape (rows, columns)
print("Shape:", df.shape)

# List all column names
print("Columns:", df.columns.tolist())

# Display first 5 rows
print(df.head())
```

Sample Output:

```
Shape: (100, 5)
Columns: ['Roll_No', 'Name', 'Age', 'Department', 'CGPA']
Roll_No    Name    Age    Department    CGPA
0          1    Alice    20          CSE    8.5
1          2     Bob    21          ECE    7.8
2          3  Charlie    19          CSE    9.1
3          4   Diana    22           ME    6.9
4          5     Eve    20          CSE    8.0
```

SECTION B — Analytical / Application Questions (8 Marks Each, Do Any 5)

Q6. k-Nearest Neighbours (k-NN) algorithm with example.

k-NN is a supervised, instance-based (lazy) learning algorithm used for classification and regression. It classifies a new data point based on the majority class of its *k* nearest neighbours.

Algorithm Steps:

- 1. Choose the value of *k* (number of neighbours).
- 2. Calculate the distance between the new point and all training points.
- 3. Select the *k* nearest neighbours (smallest distances).
- 4. Assign the class based on **majority vote** (classification) or **average** (regression).

Example: Suppose we want to classify a new fruit with Weight = 150g, Colour score = 7.

Fruit	Weight	Colour Score	Class
A	140	6	Apple
B	130	8	Apple
C	170	9	Orange
D	160	7	Orange
E	145	6.5	Apple

For *k* = 3, the 3 nearest points might be A, D, E → 2 Apple, 1 Orange → **Predicted: Apple**

Effect of *k*:

- **Small *k* (e.g., *k*=1):** Sensitive to noise and outliers; overfitting.
- **Large *k* (e.g., *k*=20):** Over-smoothing; underfitting; may ignore local patterns.
- **Best practice:** Choose odd *k* (to avoid ties), use cross-validation to find optimal *k*.

Distance Metrics:

- 1. **Euclidean Distance:** $d = \sqrt{\sum (x_i - y_i)^2}$ — Most commonly used for continuous data.
- 2. **Manhattan Distance:** $d = \sum |x_i - y_i|$ — Sum of absolute differences, better for high-dimensional data.
- 3. **Minkowski Distance:** Generalizes both Euclidean (*p*=2) and Manhattan (*p*=1).

Q7. Pandas code for retail dataset operations.

```
import pandas as pd

# Assume df is a DataFrame loaded from the retail dataset
df = pd.read_csv('retail_data.csv')

# _____
# (i) Filter products with Rating > 4.0 AND Quantity_Sold > 100
# _____
filtered_df = df[(df['Rating'] > 4.0) & (df['Quantity_Sold'] > 100)]
```

```
print("Filtered Products:")
print(filtered_df)

# -----
# (ii) Group by Category and compute average Price
# -----
avg_price = df.groupby('Category')['Price'].mean()
print("\nAverage Price by Category:")
print(avg_price)

# -----
# (iii) Sort in descending order of average price
# -----
sorted_avg_price = avg_price.sort_values(ascending=False)
print("\nSorted (Descending) Average Price:")
print(sorted_avg_price)
```

Explanation:

- `df[(condition1) & (condition2)]` — Boolean indexing to filter rows meeting both conditions.
- `df.groupby('Column')['Target'].mean()` — Groups rows by category and computes mean of a target column.
- `.sort_values(ascending=False)` — Sorts values in descending order.

Q8. Data Reduction — Three techniques with examples.

Data Reduction is the process of reducing the volume of data while maintaining its integrity, so that analysis produces similar or identical results on the reduced dataset.

Why it's important: Large datasets increase computation time, memory usage, and model complexity. Reducing data before ML leads to faster training, reduced overfitting, and more interpretable models.

Three Techniques:

1. Dimensionality Reduction: Reduces the number of features/attributes.

Method	Description
PCA (Principal Component Analysis)	Transforms data into a smaller set of uncorrelated components that capture maximum variance.
Feature Selection	Select only the most relevant features (e.g., using correlation analysis or chi-square test).

Example: A dataset with 100 features can be reduced to 10 principal components using PCA, retaining 95% of the variance.

2. Numerosity Reduction: Reduces the number of data points by using smaller representations.

Method	Description
Parametric	Store only model parameters instead of raw data (e.g., store regression coefficients).
Non-Parametric	Use techniques like sampling, histograms, or clustering to represent data.

Example: Instead of storing 1 million transaction records, fit a regression model and store the coefficients.

3. Data Compression: Encodes data to take less space.

Method	Description
Lossless	No information is lost (e.g., ZIP, gzip).
Lossy	Some information is lost, but the result is approximately correct (e.g., JPEG, wavelet transforms).

Example: Compressing a 10GB log file to 1GB using gzip (lossless) — data can be fully reconstructed.

Q9. Comparison of Histograms, Boxplots, and Scatterplots with code.

Feature	Histogram	Boxplot	Scatterplot
Purpose	Shows the distribution of a single numeric variable.	Shows the spread, median, quartiles, and outliers of a numeric variable.	Shows the relationship between two numeric variables.
Axes	X: bins (value ranges), Y: frequency	X: category (optional), Y: numeric values	X: variable 1, Y: variable 2
Best Used When	Understanding data distribution (normal, skewed, bimodal).	Comparing distributions across categories; detecting outliers.	Identifying correlations, trends, or clusters between two variables.
Key Insight	Shape of distribution	Median, IQR, outliers	Correlation (positive, negative, none)

Python Code — Boxplot using Seaborn:

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Sample data
data = {
    'Department': ['CSE']*5 + ['ECE']*5 + ['ME']*5,
    'Salary': [45000, 55000, 60000, 52000, 48000,
               40000, 42000, 38000, 44000, 41000,
               35000, 37000, 39000, 33000, 36000]
}
df = pd.DataFrame(data)

# Boxplot
plt.figure(figsize=(8, 5))
sns.boxplot(x='Department', y='Salary', data=df, palette='viridis')
plt.title('Salary Distribution by Department')
plt.xlabel('Department')
plt.ylabel('Salary (₹)')
plt.show()
```

Python Code — Histogram using Matplotlib:

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.normal(50, 10, 500) # 500 random values, mean=50, sd=10

plt.figure(figsize=(8, 5))
plt.hist(data, bins=20, color='steelblue', edgecolor='black', alpha=0.7)
plt.title('Distribution of Scores')
plt.xlabel('Score')
plt.ylabel('Frequency')
plt.show()
```

Q10. K-Means Clustering — Algorithm + Two Iterations.

K-Means Algorithm Steps:

- 1. **Choose K** — the number of clusters.

2. **Initialize centroids** — randomly or manually.
3. **Assign each point** to the nearest centroid (using Euclidean distance).
4. **Update centroids** — compute the mean of all points in each cluster.
5. **Repeat steps 3–4** until centroids do not change (convergence).

Given: Data = {2, 4, 10, 12, 3, 20, 30, 11, 25} , C1 = 2, C2 = 4

Iteration 1 — Assignment:

Point	d to C1=2	d to C2=4	Cluster
2	0	2	C1
4	2	0	C2
10	8	6	C2
12	10	8	C2
3	1	1	C1 (tie → assign to C1)
20	18	16	C2
30	28	26	C2
11	9	7	C2
25	23	21	C2

Cluster 1: {2, 3} → New C1 = $(2 + 3) / 2 = 2.5$ **Cluster 2:** {4, 10, 12, 20, 30, 11, 25} → New C2 = $(4 + 10 + 12 + 20 + 30 + 11 + 25) / 7 = 112 / 7 = 16$

Iteration 2 — Assignment (C1 = 2.5, C2 = 16):

Point	d to C1=2.5	d to C2=16	Cluster
2	0.5	14	C1
4	1.5	12	C1
10	7.5	6	C2
12	9.5	4	C2
3	0.5	13	C1
20	17.5	4	C2
30	27.5	14	C2
11	8.5	5	C2
25	22.5	9	C2

Cluster 1: {2, 4, 3} → New C1 = $(2 + 4 + 3) / 3 = 3.0$ **Cluster 2:** {10, 12, 20, 30, 11, 25} → New C2 = $(10 + 12 + 20 + 30 + 11 + 25) / 6 = 108 / 6 = 18.0$

Q11. Ethical Considerations in Data Science.

Ethical considerations are moral principles and guidelines that data scientists must follow to ensure responsible and fair use of data.

1. Data Privacy:

- Individuals have the right to know how their data is collected, stored, and used.
- **GDPR** (General Data Protection Regulation) mandates explicit consent and right to be forgotten.
- Example: Collecting location data from a fitness app without informing users violates their privacy.

2. Algorithmic Bias:

- ML models can inherit biases present in training data, leading to unfair outcomes.
- Bias can occur due to imbalanced datasets, biased labelling, or historical prejudice.
- Example: Amazon's AI recruiting tool (2018) was scrapped because it showed bias against women — the model was trained on 10 years of resumes predominantly from men.

3. Informed Consent:

- Users must be clearly informed about what data is being collected and how it will be used, before they provide it.
- Consent should be explicit, voluntary, and revocable.
- Example: Facebook-Cambridge Analytica scandal (2018) — personal data of ~87 million users was harvested without explicit consent for political profiling.

4. Transparency and Accountability:

- Algorithms should be explainable (not "black boxes").
- Organizations must be accountable for the outcomes of their data-driven decisions.

5. Data Security:

- Implementing encryption, access controls, and secure storage to prevent breaches.
- Example: Equifax data breach (2017) exposed personal data of 147 million people due to inadequate security measures.

Real-World Case — Cambridge Analytica: Cambridge Analytica accessed Facebook users' data through a third-party quiz app, profiled users based on personality traits, and used this to deliver targeted political advertisements during the 2016 US election. This led to massive public outrage, over \$5 billion in fines for Facebook, and stricter global data privacy laws.

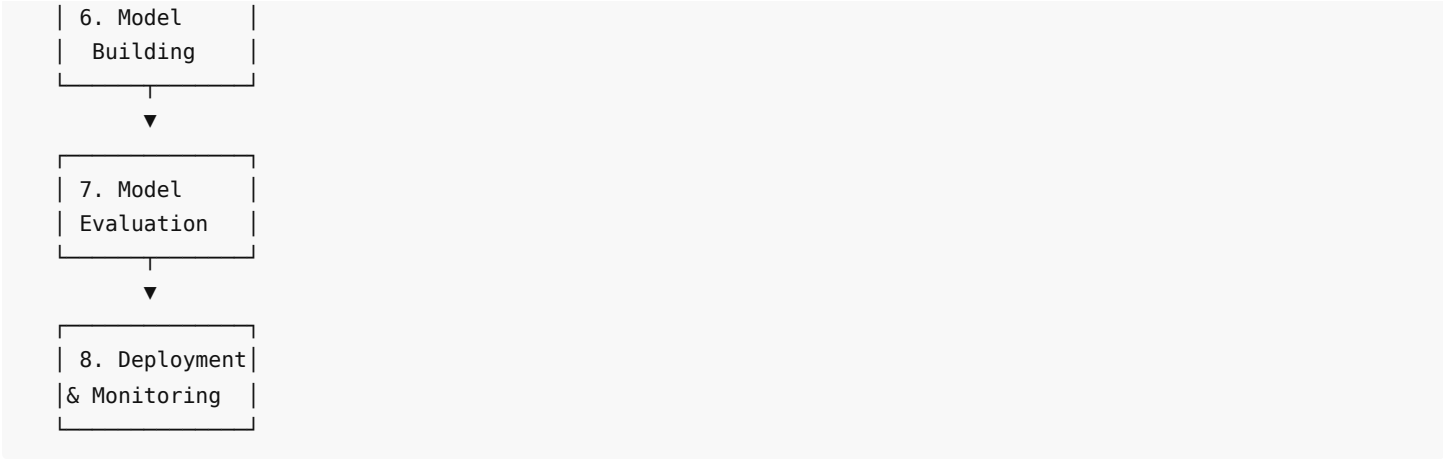
SECTION C — Long Answer / Case-Based Questions (20 Marks Each, Do Any 2)

Q12.

(a) Complete Data Science Lifecycle (10 Marks)

Data Science Lifecycle is a systematic process for extracting insights from data:





Detailed Phases:

1. Problem Definition:

- Clearly define the business problem or research question.
- Example: "Predict which customers are likely to churn in the next 3 months."

2. Data Collection:

- Gather raw data from databases, APIs, surveys, web scraping, IoT sensors.
- Example: Collecting customer transaction history, demographic data, and support tickets.

3. Data Cleaning:

- Handle missing values, outliers, duplicates, and inconsistencies.
- Example: Filling missing Age values with median, removing duplicate customer IDs.

4. Exploratory Data Analysis (EDA):

- Visualize and summarize data using statistics and plots to discover patterns.
- Compute measures of central tendency (mean, median, mode) and dispersion (range, variance, IQR).
- Example: Plotting histograms of purchase frequency, boxplots for salary distribution.

5. Feature Engineering:

- Create new features, select important features, and transform variables.
- Example: Creating Tenure_Category from Joining_Date , one-hot encoding Department .

6. Model Building:

- Choose and train ML algorithms (k-NN, K-Means, Decision Trees, Regression).
- Example: Training a Random Forest classifier on labelled churn data.

7. Model Evaluation:

- Assess performance using metrics: Accuracy, Precision, Recall, F1-Score, RMSE.
- Example: Model achieves 92% accuracy and 0.88 F1-score on test data.

8. Deployment & Monitoring:

- Deploy the model to production (API, dashboard, automated system).
- Monitor for performance degradation (data drift, concept drift).
- Example: Deploying churn prediction model as a REST API integrated with the CRM system.

(b) Five Sources of Data (10 Marks)

Source	Description	Real-World Use Case	Challenges
Time Series Data	Data collected at regular intervals over time.	Stock price prediction — historical prices collected daily.	Handling seasonality, trends, stationarity, missing timestamps.

Transactional Data	Data generated from business transactions.	Amazon purchase records — each order creates a transaction record.	High volume, real-time processing needed, data consistency across systems.
Biological Data	Data from biological/medical sources.	Genomic sequencing data used for personalized medicine and drug discovery.	Extremely large datasets, privacy concerns (patient data), complex structures (DNA sequences).
Spatial Data	Data associated with geographic locations.	Uber/Ola using GPS data for route optimization and demand forecasting.	Large volume, varying coordinate systems, real-time processing, integration of satellite imagery.
Social Network Data	Data generated from social media platforms and online interactions.	Twitter sentiment analysis during elections to gauge public opinion.	Unstructured data (text, images), noise, fake accounts/bots, privacy and ethical issues, API rate limits.

Additional Note on Data Evolution: Data evolves over time — new sources emerge (IoT, wearables), formats change, volumes grow exponentially. Data scientists must build adaptable pipelines that handle schema changes, versioning, and evolving data quality.

Q13. Complete Python program for employee_performance.csv.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# =====
# Task 1: Read CSV, display shape, data types, first 5 rows
# =====
df = pd.read_csv('employee_performance.csv')

print("Shape of dataset:", df.shape)
print("\nData Types:")
print(df.dtypes)
print("\nFirst 5 Rows:")
print(df.head())

# =====
# Task 2: Mean, Median, and Mode of Performance_Score
# =====
mean_score = df['Performance_Score'].mean()
median_score = df['Performance_Score'].median()
mode_score = df['Performance_Score'].mode()[0]

print(f"\nPerformance_Score Statistics:")
print(f"  Mean    = {mean_score:.2f}")
print(f"  Median  = {median_score:.2f}")
print(f"  Mode    = {mode_score}")

# =====
# Task 3: Missing values – count & impute
# =====
print("\nMissing Values per Column:")
print(df.isnull().sum())

# Replace missing Monthly_Salary with median
df['Monthly_Salary'] = df['Monthly_Salary'].fillna(df['Monthly_Salary'].median())

# Replace missing Performance_Score with mean
df['Performance_Score'] = df['Performance_Score'].fillna(df['Performance_Score'].mean())
```

```

print("\nMissing values after imputation:")
print(df[['Monthly_Salary', 'Performance_Score']].isnull().sum())

# =====
# Task 4: Create Salary_Category column
# =====
def categorize_salary(salary):
    if salary < 30000:
        return 'Low'
    elif salary <= 60000:
        return 'Medium'
    else:
        return 'High'

df['Salary_Category'] = df['Monthly_Salary'].apply(categorize_salary)
print("\nSalary Category Distribution:")
print(df['Salary_Category'].value_counts())

# =====
# Task 5: Boxplot and Histogram
# =====

# Boxplot – Monthly_Salary grouped by Department
plt.figure(figsize=(10, 6))
sns.boxplot(x='Department', y='Monthly_Salary', data=df, palette='Set2')
plt.title('Monthly Salary Distribution by Department')
plt.xlabel('Department')
plt.ylabel('Monthly Salary (₹)')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('salary_boxplot.png')
plt.show()

# Histogram – Performance_Score
plt.figure(figsize=(8, 5))
plt.hist(df['Performance_Score'], bins=15, color='coral',
         edgcolor='black', alpha=0.7)
plt.title('Distribution of Performance Scores')
plt.xlabel('Performance Score')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('performance_histogram.png')
plt.show()

# =====
# Task 6: IQR of Monthly_Salary and outlier detection
# =====
Q1 = df['Monthly_Salary'].quantile(0.25)
Q3 = df['Monthly_Salary'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"\nIQR Analysis for Monthly_Salary:")
print(f"   Q1 = {Q1:.2f}")
print(f"   Q3 = {Q3:.2f}")
print(f"   IQR = {IQR:.2f}")
print(f"   Lower Bound = {lower_bound:.2f}")
print(f"   Upper Bound = {upper_bound:.2f}")

```

```
outliers = df[(df['Monthly_Salary'] < lower_bound) |
              (df['Monthly_Salary'] > upper_bound)]
print(f"\nNumber of Outliers: {len(outliers)}")
print("Outlier Records:")
print(outliers[['Emp_ID', 'Name', 'Monthly_Salary']])

# =====
# Task 7: Save cleaned dataset
# =====
df.to_csv('employee_cleaned.csv', index=False)
print("\nCleaned dataset saved as 'employee_cleaned.csv'")
```

Key Concepts Used:

Concept	Description
df.shape	Returns (rows, columns) tuple
df.dtypes	Data type of each column
mean(), median(), mode()	Measures of central tendency
isnull().sum()	Count of missing values per column
fillna()	Replace NaN with a specified value
apply()	Apply a custom function to each row/column
sns.boxplot()	Create a box-and-whisker plot
plt.hist()	Create a frequency distribution histogram
quantile()	Compute percentile values for IQR
to_csv()	Export DataFrame to CSV file

IQR Method for Outlier Detection:

- **Q1** = 25th percentile, **Q3** = 75th percentile
- **IQR** = Q3 – Q1
- **Outliers** = values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$

Q14.

(a) Descriptive vs Predictive Data Mining (8 Marks)

Aspect	Descriptive Data Mining	Predictive Data Mining
Definition	Summarizes and describes past data to understand "what happened."	Uses historical data to predict future outcomes — "what will happen."
Goal	Discover patterns, correlations, and trends in existing data.	Build models that forecast future events or behaviours.
Techniques	Clustering, Association Rules, Summarization	Classification, Regression, Time Series Forecasting
Output	Reports, dashboards, pattern descriptions	Predictions, probability scores, forecasts
Example	"60% of customers who bought phones also bought cases."	"Customer X has a 78% chance of churning next month."

E-Commerce Recommendation System Case Study:

Descriptive Mining:

- **Market Basket Analysis** using association rules (Apriori algorithm).
- Discovers patterns like: *"Customers who buy laptops frequently also buy laptop bags and mice."*
- Result: Product placement strategy — display related items together.

Predictive Mining:

- **Collaborative Filtering** predicts what a user might buy based on similar users' purchase history.
- Amazon's "Customers who bought this also bought..." is powered by predictive models.
- Uses classification/regression to predict purchase probability for each item.

How They Work Together: Descriptive mining finds patterns (e.g., frequent itemsets), and predictive mining uses those patterns to make personalized recommendations, maximizing revenue and customer satisfaction.

(b) End-to-End ML Pipeline for Student Dropout Prediction (12 Marks)

Problem: Predict which students are likely to drop out, so the university can intervene early.

1. Data Collection:

- Collect data from university databases: attendance records, grades, demographic info, financial aid status, extracurricular activities, LMS (Learning Management System) engagement logs.
- Additional sources: student surveys, mentor notes, hostel records.

2. Data Preprocessing:

- **Missing Values:** Impute grades with mean/median, categorical fields with mode.
- **Encoding:** One-hot encode categorical variables (Department, Gender, Scholarship_Type).
- **Scaling:** Normalize numerical features (CGPA, Attendance%) using Min-Max or StandardScaler.
- **Data Integration:** Merge multiple data sources on Student_ID.

3. Feature Engineering:

- Create derived features:
 - Attendance_Drop = change in attendance between semesters
 - Grade_Trend = improvement or decline in grades over time
 - Financial_Stress = binary flag if student applied for fee extension
 - Engagement_Score = composite of LMS logins, assignment submissions, library visits

4. Handling Class Imbalance: Student dropout is typically a **minority class** (e.g., only 10-15% of students drop out). Techniques to handle this:

- **SMOTE** (Synthetic Minority Over-sampling Technique) — generates synthetic samples of the minority class.
- **Random Under-sampling** — reduces majority class samples.
- **Class Weights** — assign higher weight to the minority class in the loss function.
- **Ensemble Methods** — use algorithms robust to imbalance like Random Forest or XGBoost with balanced class weights.

5. Model Selection:

Recommended Approach: Supervised Learning

- This is a **binary classification** problem (Dropout: Yes/No) with labelled historical data available.
- Semi-supervised could be used if labelled data is scarce, but universities typically have historical records.
- Reinforcement Learning is not suitable as there is no sequential decision-making or reward mechanism.

Models to Try:

Model	Justification
Logistic Regression	Baseline model, interpretable
Random Forest	Handles non-linearity, robust to imbalance

Gradient Boosting (XGBoost)	High accuracy, handles missing values
k-NN	Instance-based, good for small datasets

6. Model Training:

- Split data: **80% training, 20% testing** (stratified split to preserve class ratio).
- Use **k-fold cross-validation** (k=5) for robust evaluation.
- Hyperparameter tuning using GridSearchCV or RandomizedSearchCV.

7. Model Evaluation Metrics:

Metric	Formula	Why It Matters
Accuracy	$(TP + TN) / \text{Total}$	Overall correctness, but misleading with imbalanced data.
Precision	$TP / (TP + FP)$	Of all predicted dropouts, how many actually dropped out? Reduces false alarms.
Recall (Sensitivity)	$TP / (TP + FN)$	Of all actual dropouts, how many were correctly identified? Most important — missing a dropout is costly.
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of Precision and Recall; best single metric for imbalanced data.

For this use case, **Recall is prioritized** — it's better to falsely flag a student (investigate and find they're fine) than to miss a student who actually drops out.

- Confusion Matrix** provides a detailed breakdown of TP, TN, FP, FN.
- ROC-AUC Curve** visualizes the trade-off between sensitivity and specificity.

8. Deployment:

- Deploy as a **dashboard** for academic advisors showing:
 - Risk score for each student (0-100%)
 - Key risk factors (low attendance, grade decline, financial stress)
 - Recommended interventions
- Integrate with the university's Student Information System (SIS).
- Set up **alerts** when a student's risk score exceeds a threshold (e.g., 70%).
- Monitor** model performance quarterly; retrain with new semester data.